

CSC311: Final Report

Lina Jin, Fiona Hoang, Jihyuk Yoon, Vincent Chang

April 3rd, 2025

Data Exploration and Analysis

Overview of Dataset

Before modelling, we first cleaned and conducted an exploratory data analysis (EDA) on the dataset of student responses. The dataset provided contained 1644 rows with each row corresponding to a single student's responses to questions about a particular food item. Each response includes 8 features, and a target label indicates whether the food item is **Pizza**, **Shawarma**, or **Sushi**.

We cleaned and examined the following features:

- **Q1:** Expected complexity of making the food item from a scale of 1-5 (multiple-choice)
- **Q2:** Expected number of ingredients in the food item (free-text)
- **Q3:** In what settings would they expect the food item to be served (multiple-choice)
- **Q4:** Expected price per serving (free-text)
- **Q5:** Movie associated with the food (free-text)
- **Q6:** What drink they would pair with the food item (free-text)
- **Q7:** Who the food reminds them of (multiple choice)
- **Q8:** How much hot sauce they would add (multiple choice)

Data Cleaning and Representation

Q1 (Complexity).

Respondents answered Q1 by selecting one option from a scale from 1 to 5. We left these values unchanged and replaced any missing values with -1 .

Q2 (Number of Ingredients).

Q2 was a free-text question, and responses ranged from single numbers to paragraph long explanations. We cleaned responses to Q2 by pruning text, and extracting numerical values. If a range of numbers was found, we computed the rounded average and took that as the final value. Values such as “dried sea crunchies” were counted as unknown as it could not be parsed as one singular ingredient in the cleaning code.

Q3 (Serving Setting)

Q3 was a multi-select question, and respondents were prompted to select all settings they thought the food item would typically be served. To represent this data, we created columns for each setting, and one-hot encoded the responses.

Q4 (Expected Price).

Q4 was also a free-text question, and the same procedure used to clean Q2 was applied to Q4, producing a single integer value.

Q5 (Associated Movie).

For Q5, respondents entered a wide variety of answers, from single movie titles to elaborate explanations of why they chose a certain movie. Misspellings and formatting differences between the same movies were also common. To standardize responses, we built a list of 358 elements containing all movies mentioned as a part of a response and mapped each response to a single movie franchise. If a respondent specifically answered that no movie came to mind, then the response was labelled “no”. On the other hand, unrecognized movies were labelled with “unknown”. Finally, we one-hot encoded the movie titles, producing 358 new features.

Q6 (Drink Pairings).

The process of cleaning Q6 was largely similar to Q5. Responses were first standardized to lowercase and common misspelling were corrected. We then mapped every drink to a broader category in order to. For example, both “coke” and “pepsi” were mapped to the “soda” category. Finally, we one-hot encoded every drink category, producing 13 new features.

Q7 (Associated People).

We cleaned Q7 in the same way as Q3, as it was also a multi-select question. We one-hot encoded the responses, and produced 5 new features, one for each of parents, siblings, friends, teachers and strangers.

Q8 (Hot Sauce).

Respondents answered Q1 by selecting one option from a scale of five options. We mapped responses onto an ordinal scale from 0-4.

Data Exploration and Processing

Q2, Q4 (Free-text response converted to numerical values).

We plotted a boxplot comparing Q2 and Q4 (Figure 1). From this figure, we identified extreme values for Q4 (>30), which we removed. We then standardized Q4 by subtracting its mean and dividing by its standard deviation. Post-removal and scaling, Q4's distribution became more centered, with fewer outliers. We also discovered that some responses to Q2 greatly exceeded typical ingredient counts (e.g., >15). We removed these outliers to reduce skew.

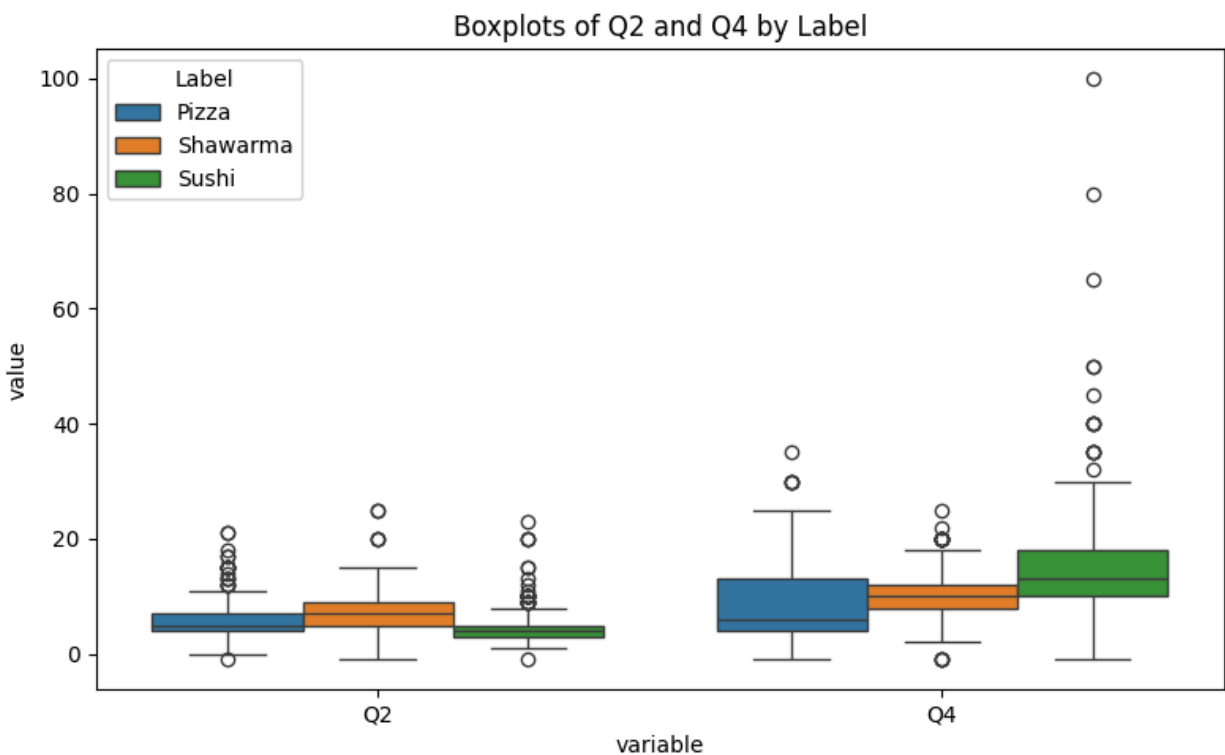


Figure 1: Boxplots of Q2 and Q4 by Label

Q1, Q2, Q4, Q8 (Numerical features)

After removing outliers and normalizing Q2 and Q4, we produced another box-plot including Q1 and Q8. The distributions of Q1, Q2, Q4 and Q8 vary significantly between each food item, which indicates that they are all informative features that could be used in our model. For example, from the distribution of Q4, we can see that respondents generally believed that pizza was the cheapest food item, with shawarma and sushi being progressively more expensive. Pizza and sushi also have lower medians for Q2 when compared to shawarma, which indicates that people generally believe there to be more ingredients in shawarma.

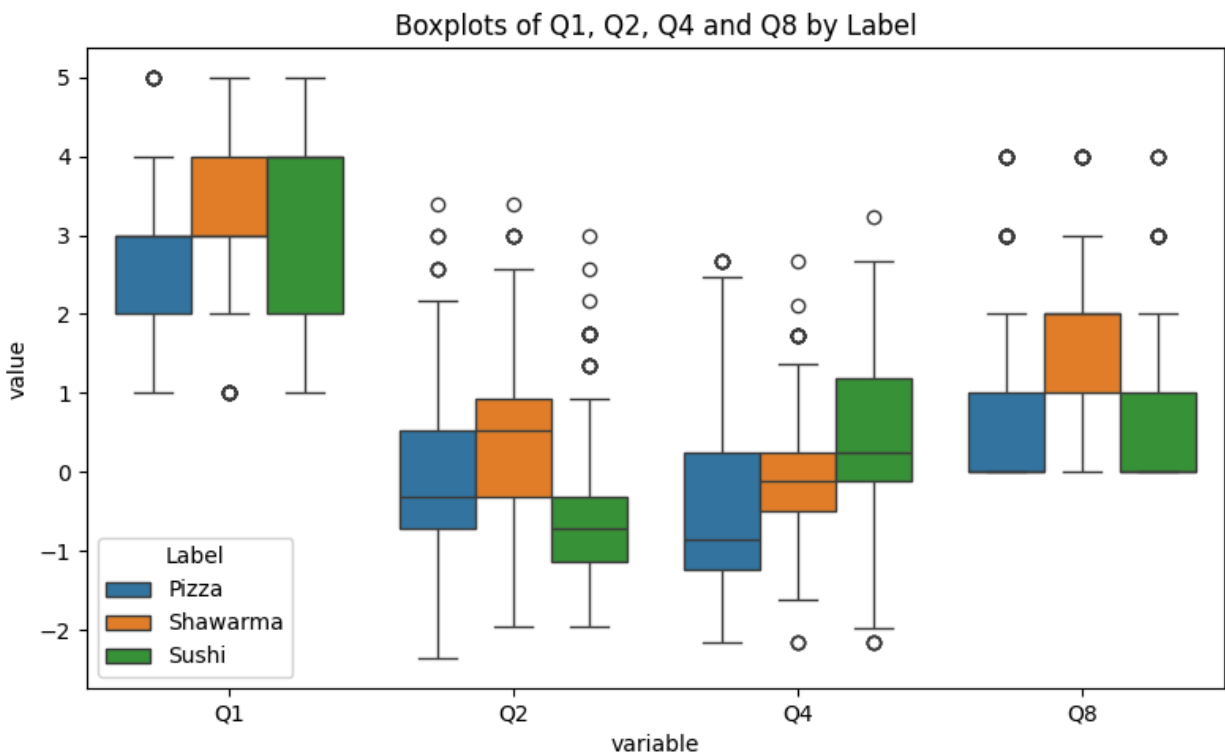


Figure 2: Boxplots of Q1, Q2, Q4, and Q8 by Label

Q3 (Serving setting)

We visualized categorical features with bar plots, separated by label. From Figure 3, we observe that pizza significantly surpasses both shawarma and sushi in terms of associations with parties. On the other hand, shawarma and sushi have high counts for weekday lunches and weekday dinners respectively, though shawarma is a particularly unpopular weekday dinner option. In general, weekend lunches and dinners seem to be more evenly distributed. Arguably, “weekday lunch”, “weekday dinner” and “at a party” are fairly informative features, while “weekend lunch” and “weekend dinner” may be less useful.

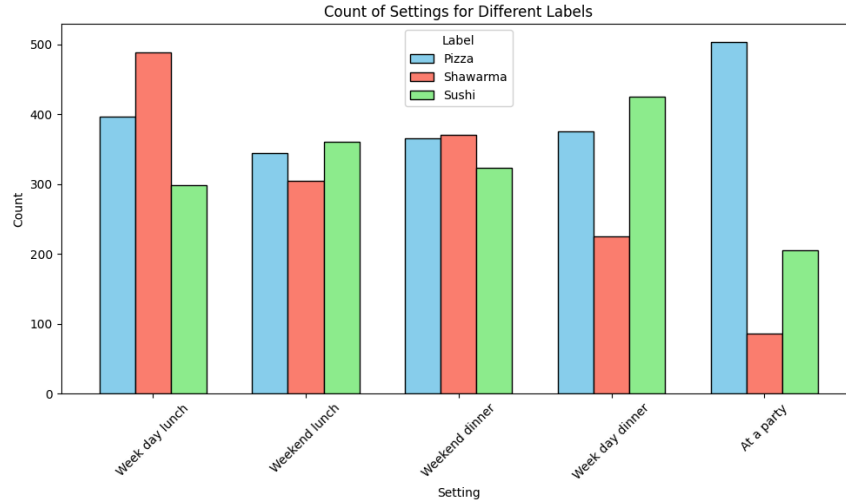


Figure 3: Histograms of Q3 by Label

Q7 (Associations with people)

Overall, Pizza seemed to be the most socially popular food, scoring high counts across most categories. People seem to associate pizza with friends, and in particular, pizza is the only food item to be significantly associated with teachers. On the other hand, shawarma seems to evoke less associations overall, but demonstrates a particularly strong association with strangers. Finally, sushi is generally associated with parents and siblings. The distributions of each label vary in unique ways, which indicates that Q7 is likely an informative feature. It is also important to note that Q3 and Q7 are both select all that apply questions, so the model can learn based on the aggregate distribution of all the categories.

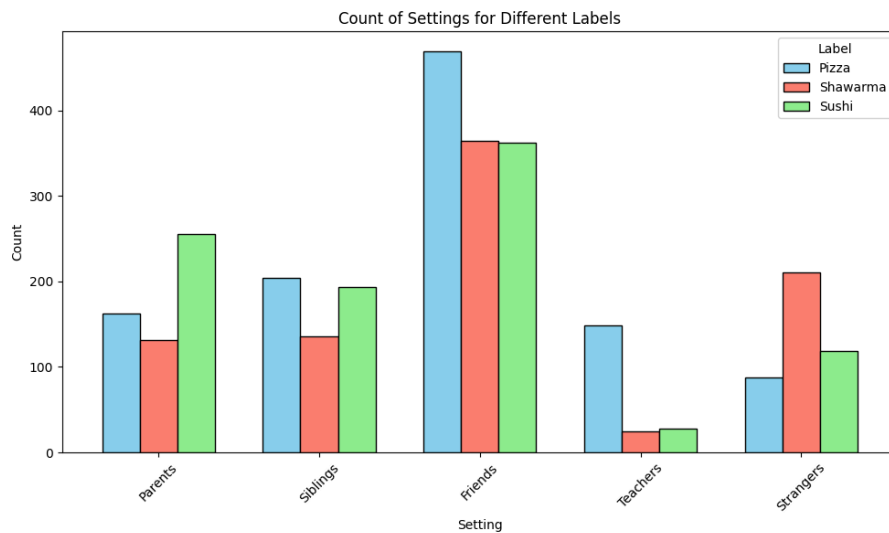


Figure 4: Histograms of Q7 by Label

Q5 (Movies)

People generally associated pizza with well-known, family friendly movies, with “Home Alone” and “Spiderman” being particularly popular choices. The movie “Avengers” was overwhelmingly associated with shawarma, with over 200 positive counts. This is likely due to a popular post-credits scene where the main characters are seen eating at a shawarma restaurant after the final showdown of the movie. The movies associated with sushi were more varied, though most of them are either set, or produced in Japan. Overall, “Jiro Dreams of Sushi” and “Spirited Away” were popular choices. It is also important to note that across all three food categories, a significant number of respondents didn’t associate the food item with any movie.

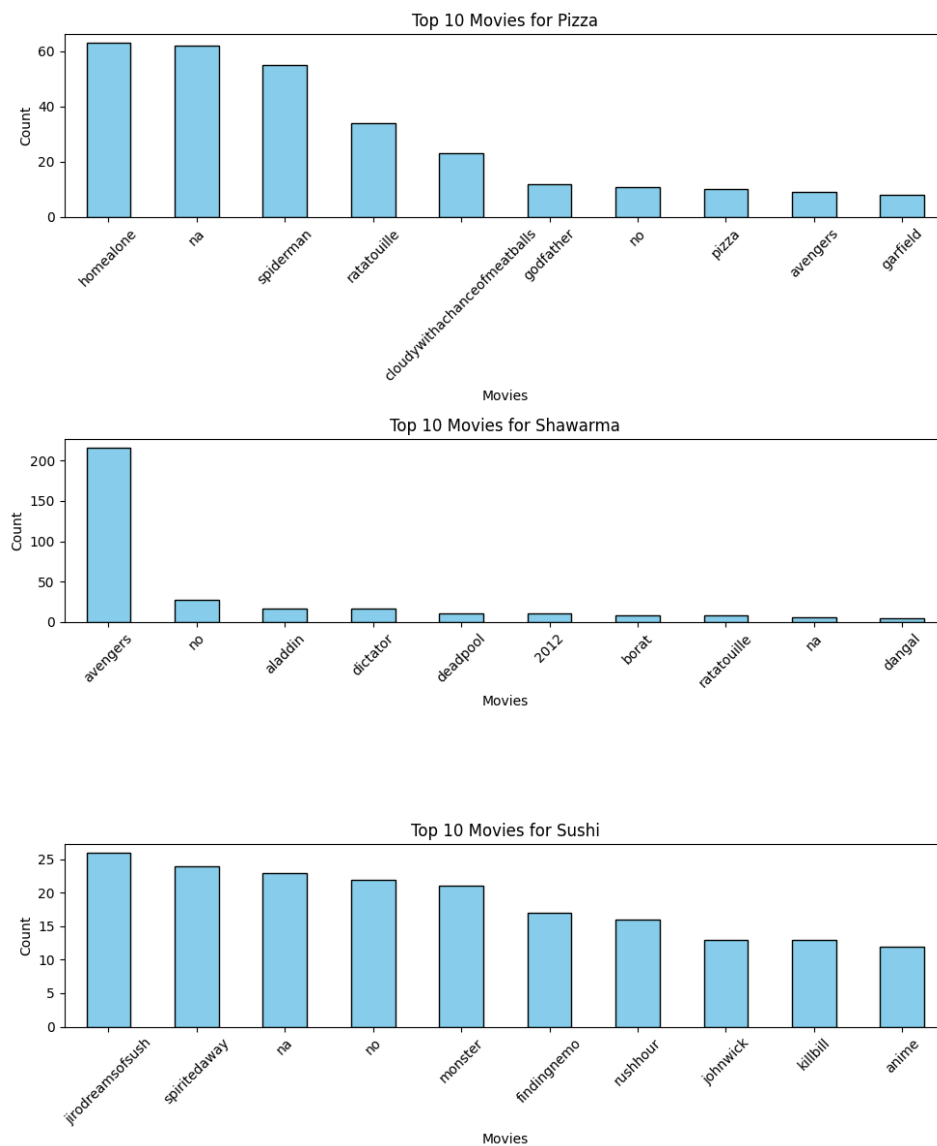


Figure 5: Histograms of Q5 by Label

Q6 (Drink Pairings)

The counts of preferred drink pairings across labels varied greatly, with soda being overwhelmingly preferred when paired with pizza. There is a slightly greater variety of drinks when paired with shawarma, though soda is still the most popular. Finally, respondents generally preferred to have their sushi with water or tea. Sushi is also the only category where asian alcohols were considered a preferred pairing.

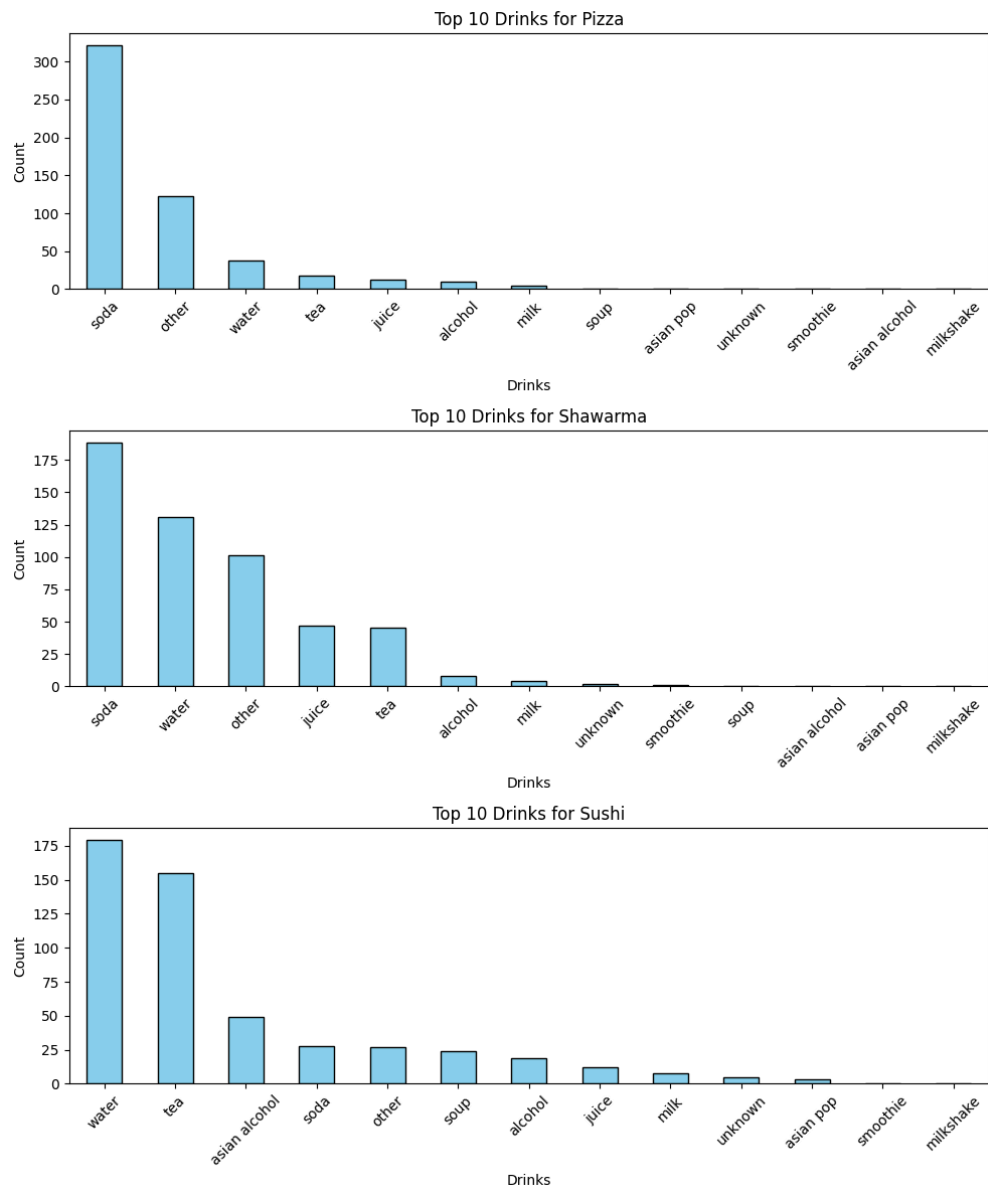


Figure 6: Histograms of Q6 by Label

Correlation Heatmap

We produced a heatmap (Figure 6) to visualize the correlation between columns. To analyse the correlation between features and labels, we created temporary one-hot encoded columns that indicated if a response was labeled with pizza, shawarma or sushi. Although Q1, Q2, and Q4 individually did not show very strong correlations to the labels, some of the one-hot indicators—particularly certain drink choices (Q6) and associated settings (Q3)—showed moderate correlations with one of the three food classes. This guided us to keep these categorical expansions in our final model. This analysis also helped us verify that none of our features were significantly co-linear, which could lead to numerical instability in our models.

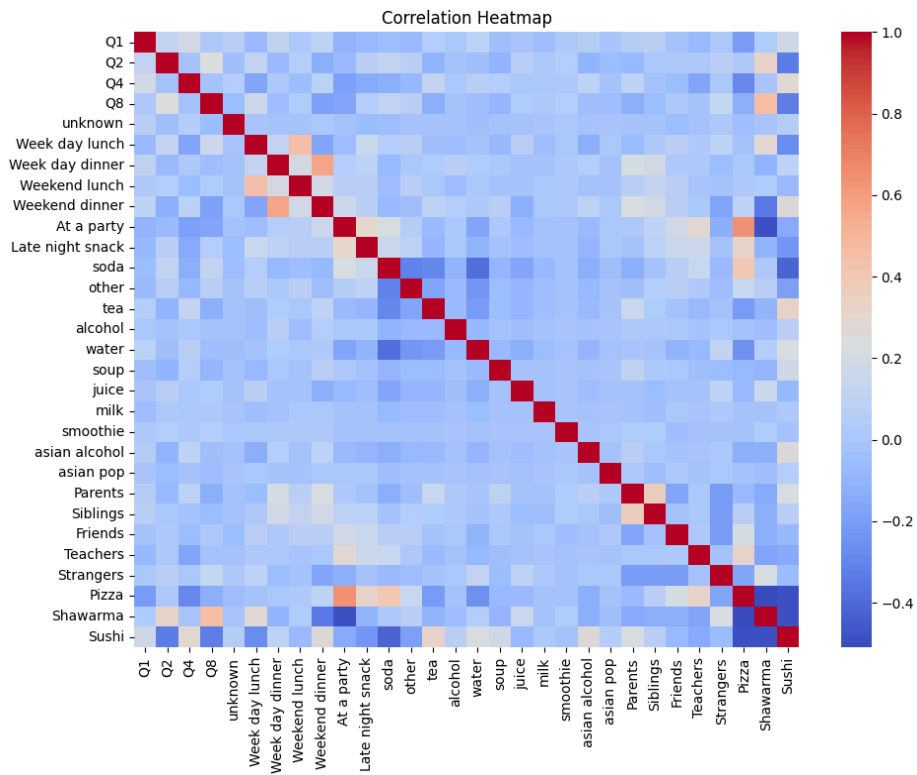


Figure 7: Pairwise Correlation Heatmap of Questions

Data Distribution and Splits

The dataset was fairly balanced among the three classes (Pizza, Shawarma, Sushi). After cleaning, pruning, and processing the provided data, our final dataset contained 1597 datapoints, with 529 data points labelled pizza and shawarma, and 509 data points labelled sushi. This helped us avoid severe class imbalance issues. Each row was made up of 385 features, with most columns being one-hot encodings of categorical features.

We split our data into 70% training, 15% validation, and 15% testing splits. This was so that 70% of our data could be enough to learn complex patterns for our neural network, while 15% could be reserved for our hyperparameter tuning, and 15% could be completely separate to ensure an honest assessment of our data and to get our estimated performance.

Models Explored and Comparisons

Given our dataset, we found that there was a non-linear relationship between variables, suggesting that a non-linear model would be best to capture the relationship between the features to predict the food. We explored three main families of models for this classification task: Naive Bayes, Decision Trees, and a custom Neural Network. In what follows, we summarize why each was chosen, how it was applied, and what performance we observed.

Naive Bayes

Rationale:

We explored Naive Bayes Classification as each feature is somewhat independent. For example, there is not a clear relationship between if you think about a teacher or not and the price of a food item, and there is no relationship between the number of ingredients needed to make this food and what movie one thinks of when they eat this food. Naïve Bayes Assumption is the assumption that each feature is independent given the class label. In our data set, each feature is independent regardless, but it is also independent given the class label too.

Further, Naive Bayes is a simple yet effective baseline for classification, especially when working with features that can be treated as conditionally independent given the class. We initially represented each response via a “bag-of-words” approach for Q1–Q8 (converted to discrete text tokens) plus a simple dictionary for mapping movies to genres. This allowed us to quickly compute likelihoods for each word/feature under each class (Pizza, Shawarma, Sushi).

Explanation:

To implement the Naïve Bayes Classification, we cleaned the data further such that we could use a method similar to the Bag of Words method. For numerical data types, such as the complexity of food, we categorized each complexity into “very simple”, “simple”, “moderate”, “complex, and “very complex”. For continuous data types such as the number of ingredients needed, we separated the numbers into 5 categories: “very few”, “few”, “moderate”, “many”, and “very many”. A similar procedure was done for categorizing price. Lastly, for the movie data, we could have left it as the movie title itself, and used each movie title as a “word”, but we further cleaned

this by making a dictionary in our Python script. We went through the data, and categorized most of the movies in the data set into different genres. Then, the script went through the data, and replaced each movie to its genre. For example, “The Dictator” was categorized as “Comedy”, and “Star Wars: The Last Jedi” was categorized as “Sci-fi, Action”.

Taking inspiration from the Naïve Bayes Classification Lab, we implemented the model. Firstly, we made the vocabulary set by going through our cleaned data. However, instead of actually taking each word as a feature, we used each feature as a feature for most of the data and separated each feature into multiple features only when there was a comma in between the words in a feature. For example, “very complex” would be left as “very complex”. However, when a feature is “Sci-fi, Action”, we separate the feature into “Sci-fi” and “Action”. This was done for two reasons. Firstly, when categorizing things such as complexity, we had options such as “very few” and “very complex”. Our Naïve Bayes Assumption is that each feature is independent of each other. However, the word “very” is dependent on what word comes afterwards. Thus, grouping them together and considering it as a feature allows us to circumvent this issue of each word being dependent on each other, and make each feature conditionally independent. Next, we split each feature with commas. This is because some questions had multiple selections, such as Question 7, where the answers would be something like “Siblings, Friends, Teachers”. In this case, each word does not have a relationship with each other like the previous example. Thinking about “Siblings” when eating sushi does not necessarily affect if one would think about “Teacher” or not when eating sushi. So, there is no direct relationship. Thus, we split each feature into separate words. The resulting vocabulary set had a size of 69 features.

Then, we augmented the bag of words function given in the lab to handle three labels. For the `make_bow(data, vocab)` function, we set the target vector t such that it outputs 0 for pizza, 1 for shawarma, and 2 for sushi. For the `naive_bayes_map(X, t)` function, we increased the output from one scalar π to three π_0 , π_1 , π_2 , corresponding to pizza, shawarma, and sushi. We also changed the shape of θ to have three columns instead of 2. Then, to calculate each θ , we used the formula for θ for MAP. Then, using the π and θ values, we augmented the `make_prediction(X,  $\pi$ ,  $\theta$ )` function to take in three π values and calculate the predictions.

Although it was able to achieve a very high training and validation accuracy, we decided not to use the Naïve Bayes classification for several reasons. Firstly, we had a problem cleaning unseen data. For example, for the number of ingredients or price, we categorized the numbers into selections based on the given data. For the number of ingredients, we set the boundary at 3, 6, 10, then 20 to determine if we should categorize it to “very few”, “few”, “moderate”, “many”, and “very many”. However, we do not know what the spread of the data would be for unseen data. Thus, using the same boundaries for arbitrary data could result in vastly different accuracy. Furthermore, when categorizing the movies, we simply used a dictionary created for the data we have. Again, the unseen data could have movie titles that are not mentioned in the data we used at all, resulting in the movie feature becoming “other”, losing a feature completely. This would affect the test accuracy as well. Thus, we decided to not move forward with this model.

Implementation Highlights:

- We formed a vocabulary of discrete tokens (size ~63 after cleaning) that included encoded words/phrases from Q1–Q8.
- Each sample was represented by a binary vector indicating presence/absence of those tokens.
- Applied **maximum a posteriori** (MAP) estimates for θ (feature probabilities) with a Beta(2,2) prior.

Performance:

From the final classification report on the validation split, Naive Bayes achieved:

- **Accuracy:** 0.806
- **F1 (macro):** 0.805

Qualitatively, the confusion matrix showed that Naive Bayes handled “pizza” fairly well but sometimes confused “shawarma” and “sushi” because some features (e.g., price or complexity) overlap. However, it still performed reasonably, confirming that many features could be treated as somewhat independent.

Decision Trees

Rationale:

We chose to investigate decision trees as they are a powerful class of universal function approximators, and are a good default option as they can process both categorical and numeric features with interpretable splits.

Explanation:

In implementing the decision trees, we used the same data cleaning and processing procedure as outlined above. However, due to the large number of features, we allowed the model to be arbitrarily deep, opting to prevent overfitting by only limiting ‘min_samples_leaf’ and ‘min_samples_split’. A significant source of potential overfitting lies in the fact that some movie titles only appear once, and are always associated with a particular food item, however, we believe restricting min_samples_leaf should work well in mitigating this issue. We tested this approach by tuning hyperparameters over across a number of combinations of ‘criterion’ (Gini,

Entropy), 'max_depth' (None, 10, 20, 30, 40, 50), 'min_samples_split' (2, 5, 10, 20, 40, 50, 60) and 'min_samples_leaf' (1, 2, 4, 8, 16). We discovered that the best hyperparameters were as follows: `criterion: 'entropy', max_depth: None, min_samples_leaf: 4, min_samples_split: 50`.

We split the data into 70% training, 15% validation and 15% test sets, and followed usual procedures of training and tuning hyperparameters before testing. The model reported 0.83 training accuracy, and 0.79 test accuracy, which suggests that it isn't overfitting on training data despite the lack of a limit on the maximum depth. Visualizing the trees also allowed us to see what the most predictive features were. In general, trees always began by checking whether the food was associated with parties, paired with soda, or associated with the movie Avengers. This aligns with our earlier analysis of the data, where we hypothesized that these features would be highly informative.

However, with a final test accuracy of 0.798, the model itself performed slightly worse when compared to neural networks and naive bayes models. We also considered the issue that training decision trees with our dataset may be computationally expensive, due to the arbitrary model depth and high-dimensional feature space.

Implementation Highlights:

- Final model parameters: `criterion: 'entropy', max_depth: None, min_samples_leaf: 4, min_samples_split: 50`
- We chose not to set a limit on the `max_depth` on the model due to a large number of features
- We were able to identify certain features with particularly high informational value

Performance:

- **Accuracy:** 0.798
- **F1 (macro):** 0.793

2-Layer Neural Network

Rationale:

Intuitively, many features themselves are contextually distinct, ranging from how many ingredients the food may contain to movies that the food is reminiscent of. The diverse array of categorical relationships of the features with the food labels provided suggest that there may be more complex relationships to analyze that a neural network could help with. Furthermore, with

thousands of lines of data, we thought a neural network would maximize the accuracy of our prediction. A feed-forward Neural Network can learn higher-level interactions given enough data and the right regularization.

Explanation:

Our final model utilized a two-layer neural network approach. The model consists of an input layer of 385 features, a hidden layer with 64 nodes in the hidden layers using Rectified Linear Unit (ReLU) activation, and an output layer with 3 classes corresponding with either Shawarma, Pizza, or Sushi. We used ReLU activation in the hidden layers as it introduces non-linearity to our model and compresses the high-dimensional feature space. The output layer applies softmax activation to produce class probabilities with L2 regularization applied to reduce overfitting. Softmax Activation was used in the output layer to transform the raw outputs to an interpretable output. Since this is a relatively simple classification task with only 3 classes, we thought that 2 layers would be sufficient for our use case.

We chose to use 64 nodes in our hidden layers since most of our features nodes correspond to single movie titles, with very sparse activations. In other words, many of the neurons in our feature layer aren't deeply informative. Therefore, a hidden layer that is significantly smaller than our feature layer may force the model to compress the high-dimensional feature space into a more informative, lower-dimensional latent-space. This would allow us to be more computationally efficient when training our model, as well as prevent overfitting since our model would have to learn more "meaningful" features in the hidden layer, instead of learning one-to-one mappings between movie titles and food categories.

Implementation Highlights:

- **Architecture:** A two-layer MLP with 64 hidden units (ReLU activation) and an output layer of 3 units (softmax). We used L2 regularization to reduce overfitting.
- **Features:** We used the fully expanded one-hot representation of Q3, Q6, Q7, Q8, plus the standardized numeric Q2, Q4, plus the Q5 genre columns. This yielded 358 features.
- **Training:** We applied mini-batch (or stochastic) gradient descent for 1000 epochs, with a learning rate of 0.01, plus a batch size of around 64 (tuned experimentally).

Performance:

- **Accuracy:** 0.855
- **F1 (macro):** 0.855

It outperformed both Naive Bayes and Decision Trees on the same train/valid split. The final classification report showed fairly balanced precision/recall across Pizza, Shawarma, and Sushi, indicating that the network learned robust, non-linear decision boundaries.

Model Comparisons

Below is a summary of the three models' performance on the validation set:

Model	Accuracy	F1 (Macro)
Neural Network	0.855	0.855
Naive Bayes	0.806	0.805
Decision Tree	0.798	0.793

The Neural Network achieved the highest F1 (macro) score and accuracy, indicating it has the best balanced performance across all three classes. Accordingly, we chose the Neural Network as our final model for the project.

Hyperparameter Tuning of the Neural Network

We decided to train our model using the mini-batch gradient descent training method, by grouping the data into mini-batches and estimating the gradient for each mini-batch until convergence. In our model, the forward pass computes activations through both layers, while the backward pass calculates the gradients for the parameters. This training method is more computationally efficient and stable in their weight measurements compared to other gradient descent methods. Our final model had a learning rate of 0.01, 1000 epochs, and a batch size of 256. This resulted in a training loss of 0.345 and validation loss 0.385. Though there is slight variability in the training loss, the loss continues to decrease towards zero, demonstrating stability with no significant signs of overfitting:

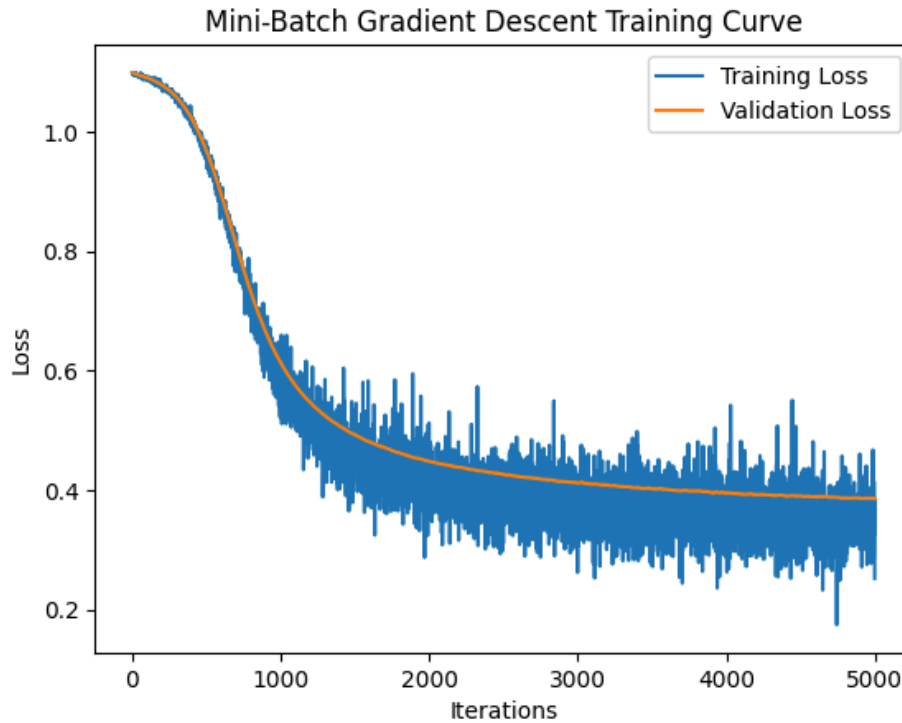


Figure 8: Training Curve of 2-Layer Neural Network with $\alpha = 0.01$, 1000 epochs, and batch size of 256

In the choice of our learning rate, we started with the standard learning rate of $\alpha = 0.01$. We further tested with values 0.001 and 0.05. We found that with a larger learning rate than 0.01 such as $\alpha = 0.05$, the training and validation loss was small, but with lots of variation, suggesting instability and possible overfitting:

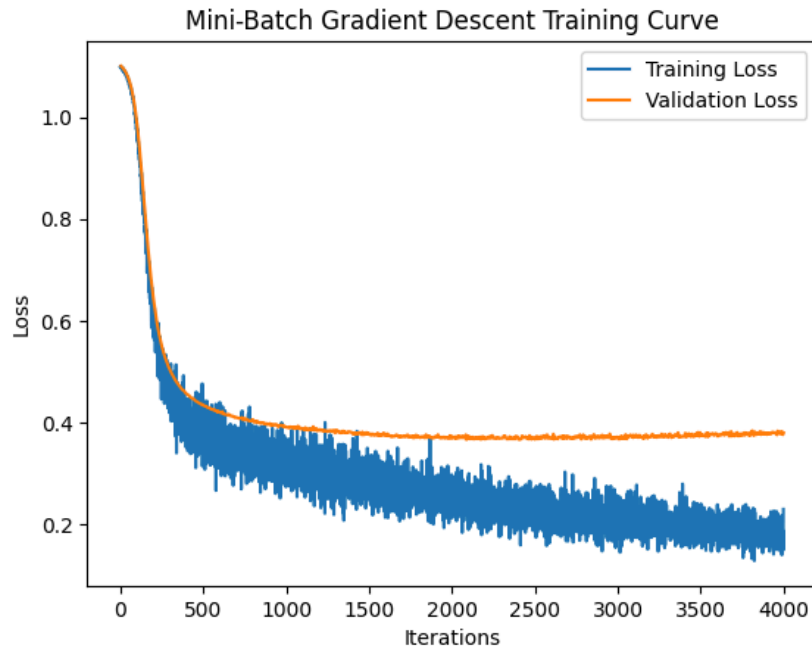


Figure 9: Training Curve of 2-Layer Neural Network with $\alpha = 0.05$

A smaller learning rate than 0.01 resulted in larger training and validation losses:

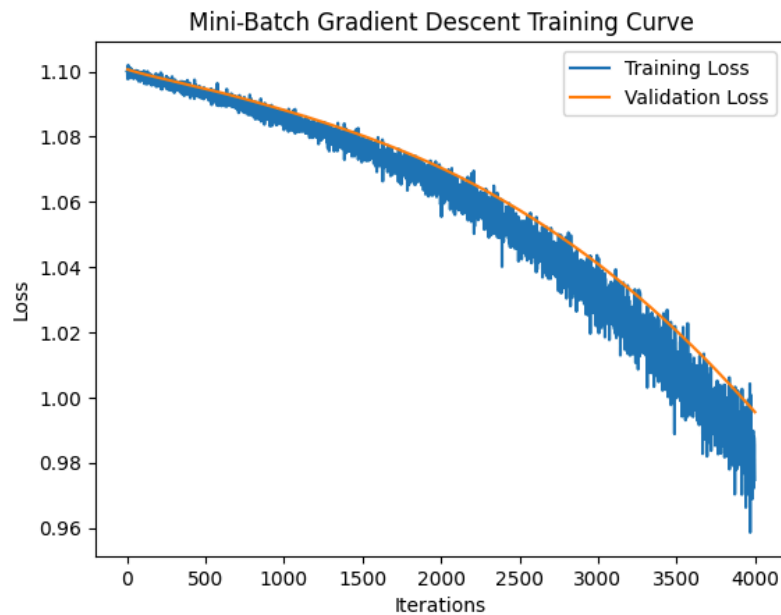


Figure 10: Training Curve of 2-Layer Neural Network with $\alpha = 0.001$

We found that $\alpha = 0.01$ is not so small that the training progresses too slowly, but not so large that it does not have time to stabilize.

For the number of epochs, we can see the validation begin to plateau at around 1000 epochs, which is different from, for example 200 epochs, which continues to decrease the loss towards zero, but results in a higher final training loss and unstable results:

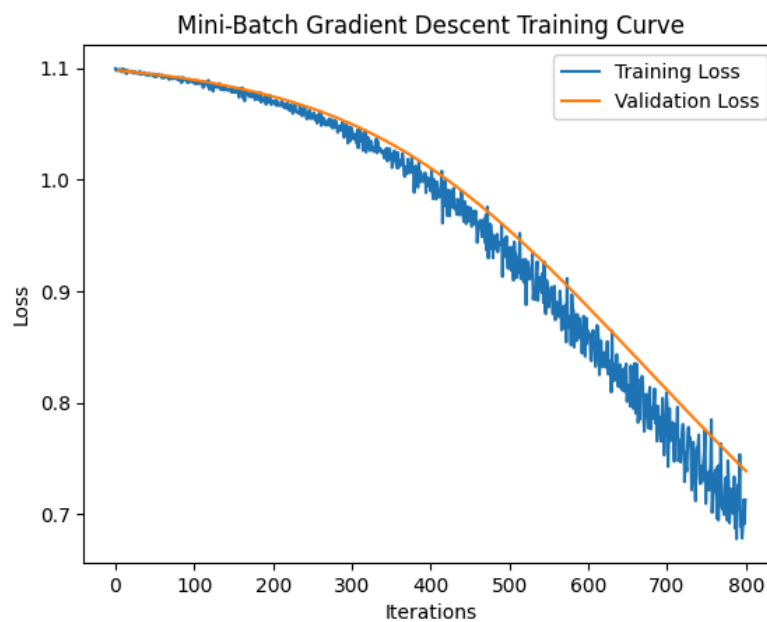


Figure 11: Training Curve of 2-Layer Neural Network with 200 Epochs

For batch size, starting out with 64, there was significant variability in the training loss.

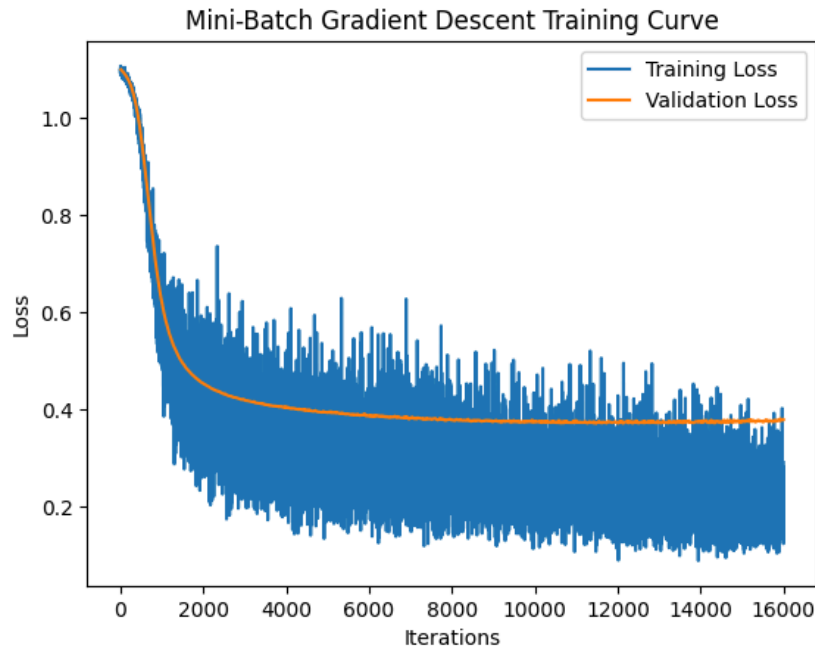


Figure 9: Training Curve of 2-Layer Neural Network with batch size 64

Increasing the batch size to 1000 stabilized the training loss.

We estimate that our model will perform with 85.5% accuracy on the test set. In order to get this number, we originally split our data with a random 70% training, 15% validation, and 15% test split on the provided data. With mini-batch testing and fine-tuning our hyperparameters of batch size, number of epochs, and learning rate, we were able to get training loss of 0.345 and validation loss 0.385, demonstrating that there are no significant signs of overfitting in our trained model. The validation loss of 0.385 demonstrates that our model is performing significantly better than random. With an evaluation metric that calculates the average of correct predictions on the validation set, we got the metric of 85.5% accuracy.

Contributions

Jihyuk Yoon

For data exploration, I wrote the script to clean the price data, and for implementation, I implemented the Naive Bayes Classification model and wrote out the explanation for the model in the report. I participated in all team meetings and contributed to deciding what models to explore and what model to choose.

Vincent Chang

I cleaned the movie response data and helped in writing/ developing visualizations for data cleaning and exploration. I was also responsible for implementing and analysing decision trees as a potential model choice.

Lina Jin

I wrote the script to clean Q2 and cleaned the data in general to normalize and remove significant outliers. I did the implementation of the neural network and helped tune its hyperparameters. I wrote the write up for the neural network implementations. I participated in the team meetings and helped finalize the documents for submission.

Fiona Hoang

For data exploration, I wrote the script to clean the drinks data (Q6). During our exploratory data analysis, I also wrote some of the code for data visualization into box plots, histograms, etc. I pooled together data cleaning code after everyone on the team has written them and combined them into one single file for ease of running and comparison. After my team members implemented their models, I wrote a comparison script comparing accuracy and F1 macro (confusion matrix) to determine the best-performing model.